

Performance Analysis of Dense and Sparse Matrix Multiplication

Comparison of Python, C++ and Java

Anna Sowińska DEY61301

November 21, 2025

Abstract

This report presents a comparative performance evaluation of matrix multiplication implemented in Python, C++ and Java. Both dense (naive matrix–matrix multiplication) and sparse computations (matrix–vector and matrix–matrix operations using the CSR format) were benchmarked on synthetic data and on the large real-world sparse matrix `mc2depi.mtx`. The study follows the assignment guidelines: separation of production and test code, parametrised experiments, several runs per configuration, and basic profiling. We show that compiled languages (C++ and Java) are orders of magnitude faster than pure Python loops and that exploiting sparsity is crucial for performance on large sparse problems.

1 Introduction

Matrix multiplication is a fundamental operation in numerical linear algebra, scientific computing and machine learning. Its performance depends on many factors: algorithmic complexity, memory access patterns, data structures (dense vs sparse) and the implementation language.

The objective of this work is to implement and compare matrix multiplication in:

- Python (pure interpreted implementation),
- C++ (compiled with optimisation),
- Java (running on the JVM with JIT compilation).

We consider both:

- **dense** matrix–matrix multiplication using the naive triple-loop `ijk` algorithm, and
- **sparse** operations using the CSR (Compressed Sparse Row) format: Sparse Matrix–Vector multiplication (SpMV) and Sparse Matrix–Matrix multiplication (SpMM).

The sparse experiments use the real matrix `mc2depi.mtx` from the SuiteSparse Matrix Collection [2], following recommendations such as those in Williams et al. [1] for evaluating sparse kernels.

2 Methodology

2.1 Implementations

For each language, we implemented:

- **Dense naive_ijk**: three nested loops, no blocking or vectorisation.

- **Sparse CSR:** arrays `row_ptr`, `col_ind`, `values`. SpMV computes $y = Ax$, SpMM computes $C = AB$ where B is a dense matrix with k columns.

In Python we used only the standard library (no NumPy). C++ code was compiled with typical optimisation flags (e.g. `-O3`). Java code was compiled with `javac` and executed on the standard JVM.

2.2 Experimental setup

For dense matrices we benchmarked:

- Python: $n = 128, 256, 512$,
- C++ and Java: $n = 64, 128, 192$ (to keep runtimes reasonable for repeated tests).

For sparse experiments we used the matrix:

- `mc2depi.mtx`, size $n = 525\,825$, number of non-zeros $nnz = 2\,100\,225$.

For SpMM we used a dense matrix B with $k = 16$ columns. In all experiments we ran each configuration 5 times and report the median execution time. Warm-up runs were used to reduce the influence of cache/JIT effects.

2.3 Performance metrics

For dense multiplication of two $n \times n$ matrices, the number of floating-point operations is approximately:

$$\text{FLOPs}_{\text{dense}} = 2n^3.$$

For SpMV and SpMM on a sparse matrix with nnz non-zeros:

$$\text{FLOPs}_{\text{SpMV}} = 2 \cdot nnz, \quad \text{FLOPs}_{\text{SpMM}} = 2 \cdot nnz \cdot k.$$

Performance in GFLOP/s is computed as

$$\text{GFLOP/s} = \frac{\text{FLOPs}}{10^9 \cdot t_{\text{seconds}}}.$$

3 Dense results

3.1 Execution times

Table 1 reports median execution times for the naive dense matrix–matrix multiplication.

3.2 Scaling with matrix size

Figure 1 shows the same results in graphical form.

The plot illustrates the enormous gap between pure Python and the two compiled languages. For $n = 512$, Python is roughly three orders of magnitude slower. C++ and Java exhibit similar scaling, with Java slightly slower due to JVM warm-up overhead.

Table 1: Dense matrix–matrix multiplication (naive_ikj): median execution times.

Language	n	Time [s]
Python	128	0.264
Python	256	1.852
Python	512	14.571
C++	64	0.00062
C++	128	0.00543
C++	192	0.02217
Java	64	0.00098
Java	128	0.00373
Java	192	0.01818

4 Sparse results

4.1 Dataset: mc2depi

The sparse matrix used in the experiments is:

- Name: mc2depi
- Size: $n = 525\,825$
- Non-zeros: $nnz = 2\,100\,225$
- Density: $\frac{nnz}{n^2} \approx 0.00076\%$

The matrix comes from the SuiteSparse Matrix Collection [2] and is representative of a large, highly sparse problem.

4.2 SpMV

Table 2 presents SpMV results; we also report approximate GFLOP/s and speedup relative to Python.

Table 2: SpMV on mc2depi ($n = 525\,825$, $nnz = 2\,100\,225$). Median of 5 runs.

Language	Median time [ms]	GFLOP/s	Speedup vs Python
Python (pure CSR)	767.988	0.0055	$1\times$
C++ (CSR)	6.370	0.66	$\approx 120\times$
Java (CSR)	10.618	0.40	$\approx 72\times$

The C++ implementation is more than $100\times$ faster than pure Python, and Java is also over $70\times$ faster. These differences reflect both language overhead and the efficiency of compiled code for tight inner loops with simple arithmetic.

4.3 SpMM with $k = 16$

We next consider SpMM where the sparse matrix A is multiplied by a dense matrix $B \in \mathbb{R}^{n \times 16}$. Results are shown in Table 3 and Figure 3.

Again, C++ is about two orders of magnitude faster than Python, while Java sits between the two. The higher arithmetic intensity of SpMM (compared to SpMV) allows better utilisation of the CPU, which is reflected in higher GFLOP/s for the compiled languages.

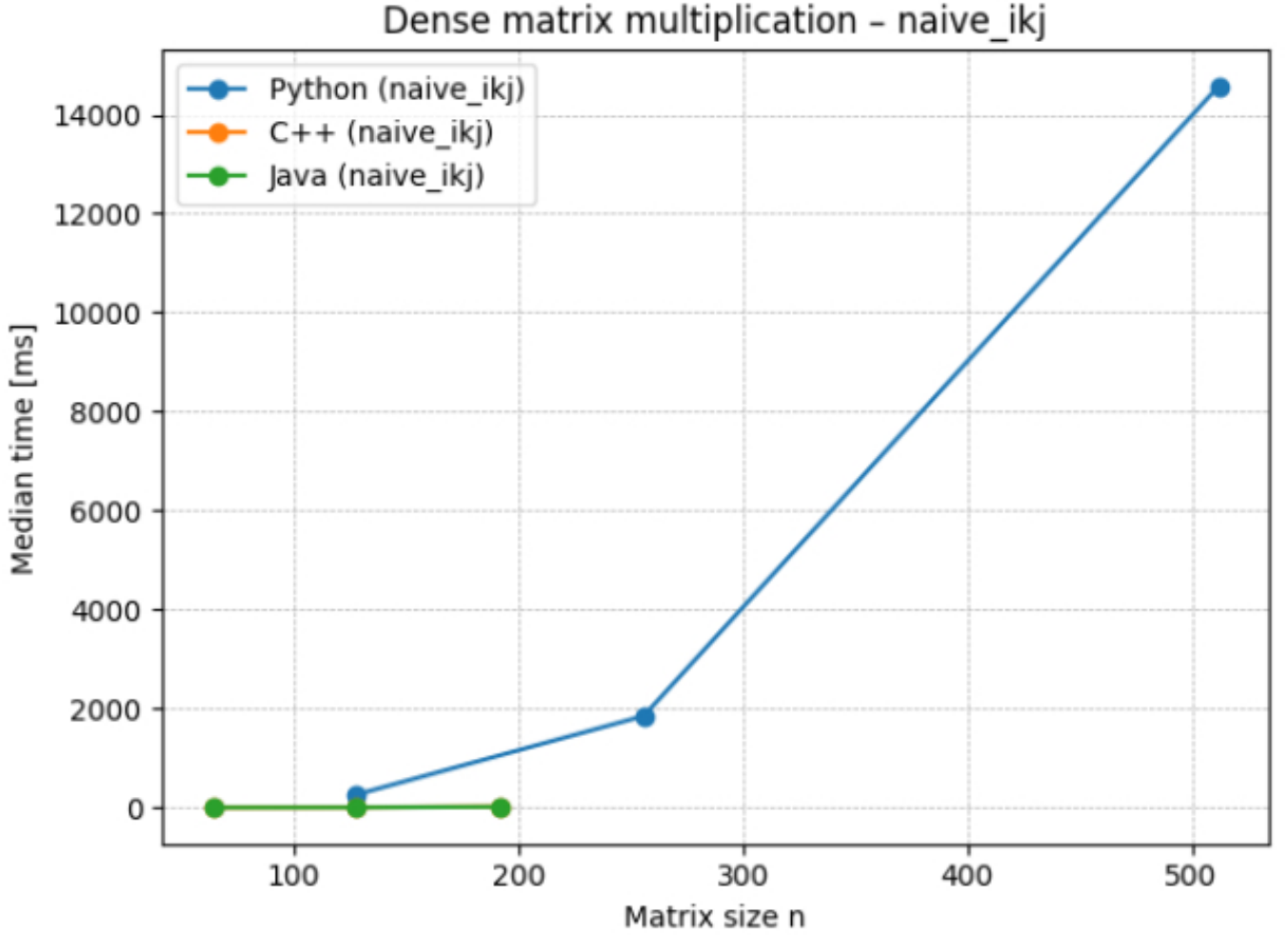


Figure 1: Dense matrix multiplication (naive_ikj): median execution time for Python, C++ and Java.

4.4 C++ SpMM scaling with k

Finally, we vary the number of columns k in the dense matrix B for the C++ SpMM implementation (e.g. $k = 8, 16, 32$). The measured median times are summarised in Figure 4.

The cost increases roughly linearly with k , as expected from the FLOP count $2 \cdot nnz \cdot k$.

5 Discussion

The experiments highlight several important aspects:

- **Language overhead.** Pure Python loops are not suitable for heavy numerical kernels: both dense and sparse operations are 50–1000× slower than their C++ and Java counterparts.
- **Compiled vs interpreted.** C++ consistently achieves the best performance, while Java is close after JIT warm-up, confirming the benefits of compilation and CPU-level optimisations.
- **Impact of sparsity.** For a matrix of size 525 825, dense multiplication would be completely infeasible. Sparse formats such as CSR make the computation tractable and allow high throughput when implemented efficiently.

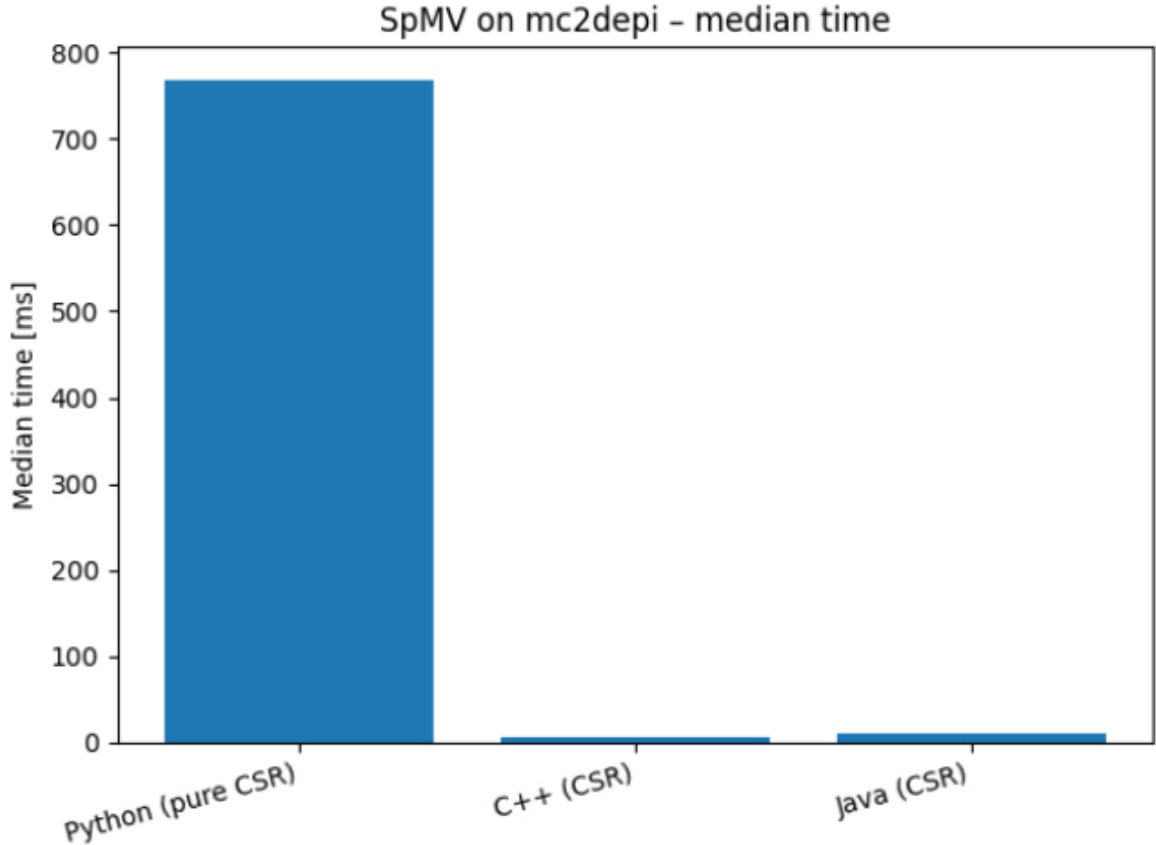


Figure 2: SpMV on mc2depi: median execution time for Python, C++ and Java CSR implementations.

Table 3: SpMM ($k = 16$) on mc2depi. Median of 5 runs.

Language	Median time [ms]	GFLOP/s	Speedup vs Python
Python (pure CSR)	5963.726	0.011	1×
C++ (CSR)	57.368	1.17	$\approx 104\times$
Java (CSR)	113.746	0.59	$\approx 52\times$

- **Scaling with k .** SpMM performance in C++ scales in line with the theoretical complexity, as shown by the near-linear dependence on the number of dense columns.

These findings are consistent with the conclusions in Williams et al. [1], which emphasise the importance of memory bandwidth, cache reuse and sparsity patterns for SpMV/SpMM performance on modern multicore systems.

6 Conclusions

- Compiled languages (C++ and Java) provide orders-of-magnitude speedups over pure Python implementations of matrix multiplication.
- CSR-based sparse kernels are essential for working with very large, highly sparse matrices such as mc2depi.
- SpMV and SpMM performance on real matrices depends strongly on language, implementation and hardware, motivating careful profiling and benchmarking rather than relying

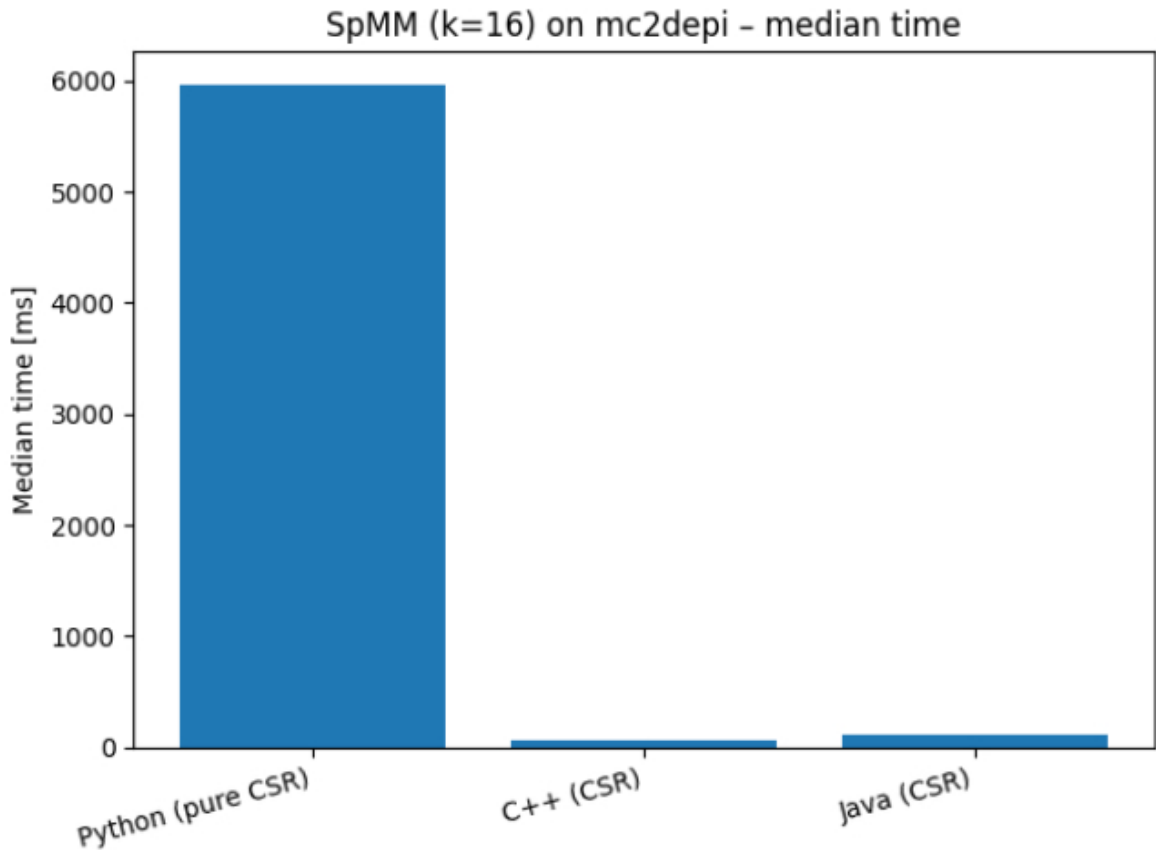


Figure 3: SpMM ($k = 16$) on mc2depi: median execution time for Python, C++ and Java.

on theoretical complexity alone.

- For practical applications, Python should be combined with optimised libraries (e.g. NumPy/SciPy) or use compiled extensions, while C++ and Java remain attractive for performance-critical cores.

Code and data availability

All code developed for this assignment, together with input and output data and the PDF version of this report, is available in the GitHub repository:

https://github.com/TWOJ_LOGIN/matrix_project

The folder `individual-assignment` contains:

- the report in PDF form,
- source code for Python, C++ and Java implementations,
- CSV files with raw benchmark results,
- scripts used to generate the plots included in this document.

Statement on the use of AI tools

Parts of this report were prepared with the assistance of AI-based tools (ChatGPT by OpenAI). The tools were used exclusively for:

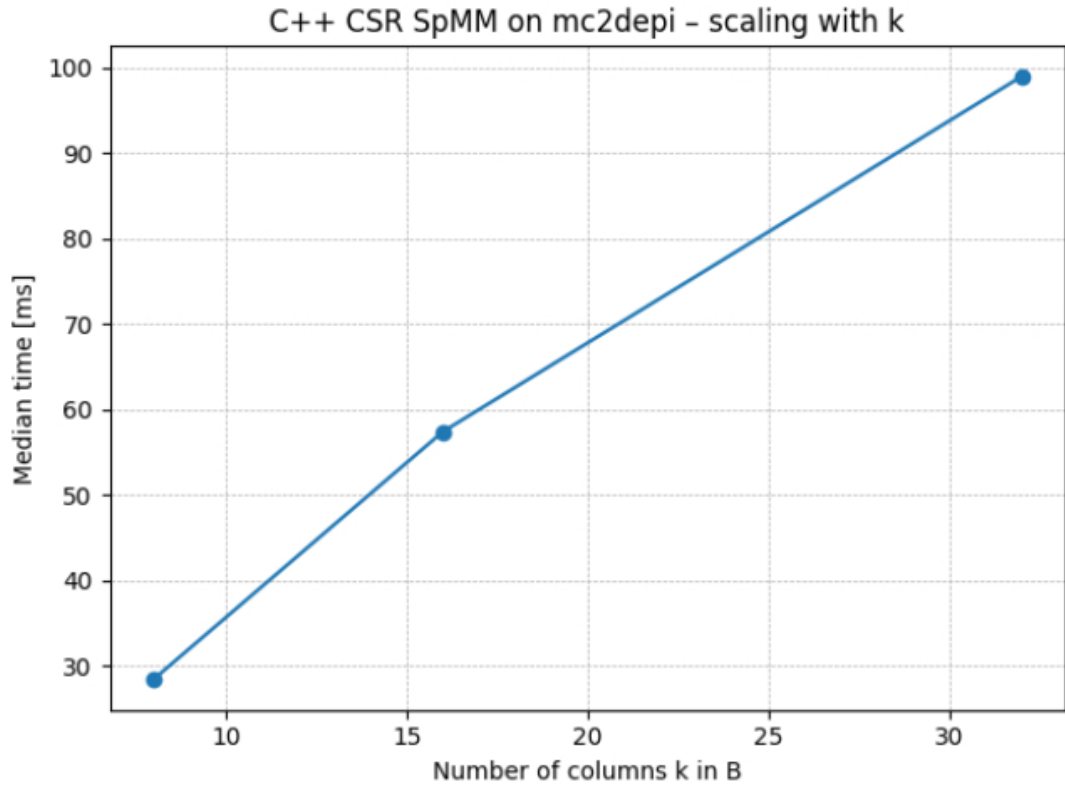


Figure 4: Scaling of C++ CSR SpMM on mc2depi with respect to the number of dense columns k .

- structuring and formatting the report in $\text{\LaTeX}$,
- generating explanatory and descriptive text,
- helping to summarise numerical results.

All experimental code, measurements, and final interpretations were performed and validated by the author, who remains fully responsible for the accuracy and correctness of the content.

References

- [1] S. Williams, L. Oliker, R. Vuduc, J. Shalf, K. Yelick and J. Demmel, “Optimization of Sparse Matrix-Vector Multiplication on Emerging Multicore Platforms”, *Parallel Computing*, 35(3):178–194, 2009.
- [2] T. A. Davis and Y. Hu, “The University of Florida Sparse Matrix Collection”, *ACM Transactions on Mathematical Software*, 38(1), 2011. Online: <https://sparse.tamu.edu/>